Analysis & Design of Algorithms Lab		Semester	4
Course Code	**BCSL404**	CIE Marks	50
Teaching Hours/Week (L:T:P: S)	0:0:2:0	SEE Marks	50
Credits	01	Exam Hours	2
Examination type (SEE)	Practical		

Course objectives:

- To design and implement various algorithms in C/C++ programming using suitable development tools to address different computational challenges.
- To apply diverse design strategies for effective problem-solving.
- To Measure and compare the performance of different algorithms to determine their efficiency and suitability for specific tasks.

Sl.No	Experiments
1	Design and implement C/C++ Program to find Minimum Cost Spanning Tree of a given connected undirected graph using Kruskal's algorithm.
2	Design and implement C/C++ Program to find Minimum Cost Spanning Tree of a given connected undirected graph using Prim's algorithm.
3	a. Design and implement C/C++ Program to solve All-Pairs Shortest Paths problem using Floyd's algorithm. b. Design and implement C/C++ Program to find the transitive closure using Warshal's algorithm.
4	Design and implement C/C++ Program to find shortest paths from a given vertex in a weighted connected graph to other vertices using Dijkstra's algorithm.
5	Design and implement C/C++ Program to obtain the Topological ordering of vertices in a given digraph.
6	Design and implement C/C++ Program to solve 0/1 Knapsack problem using Dynamic Programming method.
7	Design and implement C/C++ Program to solve discrete Knapsack and continuous Knapsack problems using greedy approximation method.
8	Design and implement C/C++ Program to find a subset of a given set S = {sl , s2,.....,sn} of n positive integers whose sum is equal to a given positive integer d.
9	Design and implement C/C++ Program to sort a given set of n integer elements using Selection Sort method and compute its time complexity. Run the program for varied values of n> 5000 and record the time taken to sort. Plot a graph of the time taken versus n. The elements can be read from a file or can be generated using the random number generator.
10	Design and implement C/C++ Program to sort a given set of n integer elements using Quick Sort method and compute its time complexity. Run the program for varied values of n> 5000 and record the time taken to sort. Plot a graph of the time taken versus n. The elements can be read from a file or can be generated using the random number generator.
11	Design and implement C/C++ Program to sort a given set of n integer elements using Merge Sort method and compute its time complexity. Run the program for varied values of n> 5000, and record the time taken to sort. Plot a graph of the time taken versus n. The elements can be read from a file or can be generated using the random number generator.
12	Design and implement C/C++ Program for N Queen's problem using Backtracking.

1. **Design and implement C/C++ Program to find Minimum Cost Spanning Tree of a given connected undirected graph using Kruskal's algorithm.**

```
#include <stdio.h>

int main() {
   int n, i, j, min, a, b, u, v, cost[20][20], visited[20];
   int ne = 1, mincost = 0;

   printf("Enter the number of vertices: ");
   scanf("%d", &n);

   printf("Enter the cost adjacency matrix:\n");
   for(i = 0; i < n; i++) {
      for(j = 0; j < n; j++) {
         scanf("%d", &cost[i][j]);
      }
    Printf("\n");
   }

   // Initialize visited array to track which nodes are included in MST
   for(i = 0; i < n; i++) {
      visited[i] = 0;  // 0 means not visited
   }

   visited[0] = 1;  // Start from the first vertex
   printf("\nThe edges of the spanning tree are:\n");

   while(ne < n) {
      min = 999;

      // Find the minimum weight edge connecting a visited node to an unvisited node
      for(i = 0; i < n; i++) {
         if(visited[i]) {
            for(j = 0; j < n; j++) {
               if(!visited[j] && cost[i][j] < min && cost[i][j] != 0) {
                  min = cost[i][j];
                  a = i;
                  b = j;
               }
            }
         }
      }
```

```
        // Add the edge to the MST
        visited[b] = 1;
        printf("Edge %d: (%d -> %d) = %d\n", ne++, a, b, min);
        mincost += min;

        // Remove the edge from consideration
        cost[a][b] = cost[b][a] = 999;
    }

    printf("\nMinimum cost = %d\n", mincost);

    return 0;
}
```

OUTPUT

2. Design and implement C/C++ Program to find Minimum Cost Spanning Tree of a given connected undirected graph using Prim's algorithm.

```
#include<stdio.h>


int a, b, u, v, n, i, j, ne = 1;
int visited[10] = {0}, min, mincost = 0, cost[10][10];

void main()
{
   printf("\n Enter the number of nodes: ");
   scanf("%d", &n);

   printf("\n Enter the adjacency matrix:\n");
   for (i = 1; i <= n; i++) {
      for (j = 1; j <= n; j++) {
         scanf("%d", &cost[i][j]);
         if (cost[i][j] == 0)
            cost[i][j] = 999;
      }
   }

   visited[1] = 1;
   printf("\n");

   while (ne < n)
  {
      for (i = 1, min = 999; i <= n; i++)
          {
         for (j = 1; j <= n; j++)
                    {
            if (cost[i][j] < min)
                              {
               if (visited[i] != 0)
                {
                  min = cost[i][j];
                  a = u = i;
                  b = v = j;
               }
          }
      }
}
```

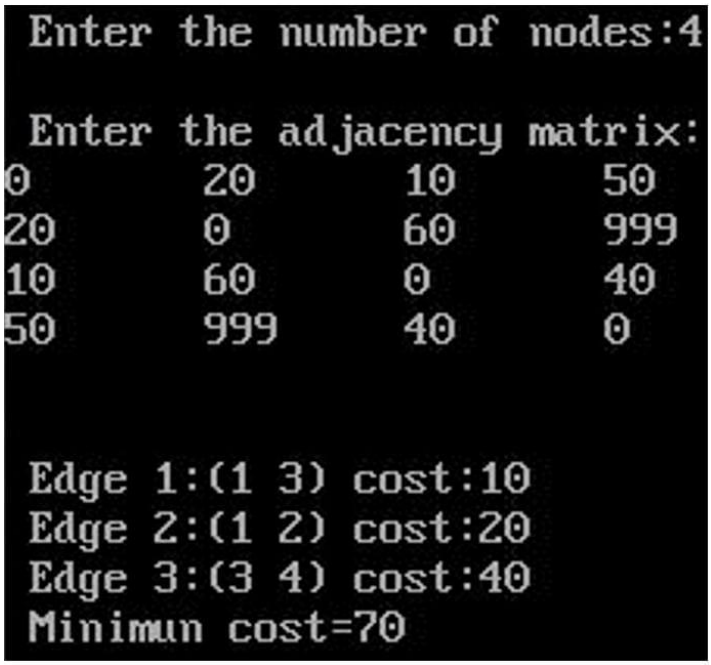
```
      if (visited[u] == 0 || visited[v] == 0)
      {
         printf("\n Edge %d:(%d %d) cost:%d", ne++, a, b, min);
         mincost += min;
         visited[b] = 1;
      }

      cost[a][b] = cost[b][a] = 999;
   }

   printf("\n Minimum cost=%d\n", mincost);

}
```

OUTPUT



3. A. Design and implement C/C++ Program to solve All-Pairs Shortest Paths problem using Floyd's algorithm.

```
#include <stdio.h>
#define INF 999

int min(int a, int b)
{
   return (a < b) ? a : b;
}

void floyd(int p[10][10], int n)
{
   int i, j, k;
   for (k = 1; k <= n; k++)
   {
      for (i = 1; i <= n; i++)
      {
         for (j = 1; j <= n; j++)
         {
            p[i][j] = min(p[i][j], p[i][k] + p[k][j]);
         }
      }
   }
}

void main() {
   int a[10][10], n, i, j;

   printf("\nEnter the n value: ");
   scanf("%d", &n);

   printf("\nEnter the graph data:\n");
   for (i = 1; i <= n; i++)
   {
      for (j = 1; j <= n; j++)
      {
         scanf("%d", &a[i][j]);
      }
   }

   floyd(a, n);

   printf("\nShortest path matrix\n");
```

```
    for (i = 1; i <= n; i++)
    {
        for (j = 1; j <= n; j++)
        {
            printf("%d ", a[i][j]);
        }
        printf("\n");
    }
}
```

Output:

```
Enter the n value: 4

Enter the graph data:

 0    999     3     999

 2      0    999    999

 999   7      0     1

 6     999   999   0


 Shortest path matrix

 0   10   3   4

 2    0   5   6

 7    7   0   1

 6    16  9   0
```

b. Design and implement C/C++ Program to find the transitive closure using Warshal's algorithm.

```
#include <stdio.h>
void warsh(int p[10][10], int n) {
    int i, j, k;
    for (k = 1; k <= n; k++) {
        for (i = 1; i <= n; i++)  {
            for (j = 1; j <= n; j++)
             {
                p[i][j] = p[i][j] || (p[i][k] && p[k][j]);
            }
        }
    }
}
void main() {
    int a[10][10], n, i, j;
    printf("\nEnter the n value: ");
    scanf("%d", &n);

    printf("\nEnter the graph data:\n");
    for (i = 1; i <= n; i++) {
        for (j = 1; j <= n; j++) {
            scanf("%d", &a[i][j]);
        }
    }
warsh(a, n);
printf("\nResultant path matrix:\n");
```

```
    for (i = 1; i <= n; i++) {
        for (j = 1; j <= n; j++) {
            printf("%d ", a[i][j]);
        }
        printf("\n");
    }
}
```

OUTPUT:

Enter the n value: 4

Enter the graph data:

0 1 0 0

0 0 0 1

0 0 0 0

1 0 1 0

Resultant path matrix:

1 1 1 1

1 1 1 1

0 0 0 0

1 1 1 1

4. Design and implement C/C++ Program to find shortest paths from a given vertex in a weighted connected graph to other vertices using Dijkstra's algorithm

```
#include <stdio.h>
#define INF 999

void dijkstra(int c[10][10], int n, int s, int d[10]) {
   int v[10], min, u, i, j;
   for (i = 1; i <= n; i++) {
      d[i] = c[s][i];
      v[i] = 0;
   }
   v[s] = 1;

   for (i = 1; i < n; i++) {
      min = INF;
      for (j = 1; j <= n; j++) {
         if (v[j] == 0 && d[j] < min) {
            min = d[j];
            u = j;
         }
      }
      v[u] = 1;
      for (j = 1; j <= n; j++) {
         if (v[j] == 0 && (d[u] + c[u][j] < d[j])) {
            d[j] = d[u] + c[u][j];
         }
      }
   }
}

int main() {
   int c[10][10], d[10], i, j, s, n;
   printf("\nEnter n value: ");
   scanf("%d", &n);
   printf("\nEnter the graph data:\n");
   for (i = 1; i <= n; i++) {
      for (j = 1; j <= n; j++) {
         scanf("%d", &c[i][j]);
      }
   }
   printf("\nEnter the source node: ");
   scanf("%d", &s);
   dijkstra(c, n, s, d);
```

```
        printf("\nShortest distances from source node %d:\n", s);
        for (i = 1; i <= n; i++) {
            printf("Node %d: %d\n", i, d[i]);
        }

    }
```

OUTPUT

Enter n value: 6

Enter the graph data:

0 15 10 999 45 999

999 0 15 999 20 999

20 999 0 20 999 999

999 10 999 0 35 999

999 999 999 30 0 999

999 999 999 4 999 0

Enter the source node: 2

Shortest distances from source node 2:

Node 1: 35

Node 2: 0

Node 3: 15

Node 4: 35

Node 5: 20

Node 6: 999

5. Design and implement C/C++ Program to obtain the Topological ordering of vertices in a given digraph

```
#include<stdio.h>

int temp[10], k = 0;

void sort(int a[10][10], int id[10], int n)
{
   int i, j;
   for (i = 1; i <= n; i++)
   {
      if (id[i] == 0)
      {
         id[i] = -1;
         temp[++k] = i;
         for (j = 1; j <= n; j++)
         {
            if (a[i][j] == 1 && id[j] != -1)
               id[j]--;
         }
         i = 0;
      }
   }
}

void main()
{
   int a[10][10], id[10], n, i, j;

   // clrscr();
   printf("\nEnter the n value: ");
   scanf("%d", &n);

   for (i = 1; i <= n; i++)
      id[i] = 0;

   printf("\nEnter the graph data:\n");
   for (i = 1; i <= n; i++)
      for (j = 1; j <= n; j++)
      {
         scanf("%d", &a[i][j]);
         if (a[i][j] == 1)
            id[j]++;
      }
```

```
        sort(a, id, n);

        if (k != n)
            printf("\nTopological ordering not possible");
        else
        {
            printf("\nTopological ordering is: ");
            for (i = 1; i <= k; i++)
                printf("%d ", temp[i]);
        }

    }
```

OUTPUT:

Enter the n value: 6

Enter the graph data:

0 0 1 1 0 0

0 0 0 1 1 0

0 0 0 1 0 1

0 0 0 0 0 1

0 0 0 0 0 1

0 0 0 0 0 0

Topological ordering is: 1 2 3 4 5 6

6. Design and implement C/C++ Program to solve 0/1 Knapsack problem using Dynamic Programming method.

```
#include <stdio.h>

int w[10], p[10], n;

int max(int a, int b) {
   return a > b ? a : b;
}

int knap(int i, int m) {
   if (i == n) return w[i] > m ? 0 : p[i];
   if (w[i] > m) return knap(i + 1, m);
   return max(knap(i + 1, m), knap(i + 1, m - w[i]) + p[i]);
}

void main() {
   int m, i, max_profit;

   printf("\nEnter the no. of objects: ");
   scanf("%d", &n);

   printf("\nEnter the knapsack capacity: ");
   scanf("%d", &m);

   printf("\nEnter profit followed by weight:\n");
   for (i = 1; i <= n; i++)
      scanf("%d %d", &p[i], &w[i]);

   max_profit = knap(1, m);

   printf("\nMax profit = %d\n", max_profit);


}
```

OUTPUT-1

Enter the no. of objects: 5

Enter the knapsack capacity: 6

Enter profit followed by weight:
78 2
45 3
92 4
71 5
56 6

Max profit = 170

OUTPUT-2

Enter the no. of objects: 5

Enter the knapsack capacity: 6

Enter profit followed by weight:
56 8
89 5
67 4
59 3
78 3

Max profit = 137

7. Design and implement C/C++ Program to solve discrete Knapsack and continuous Knapsack problems using greedy approximation method.

```
#include <stdio.h>
#define MAX 50

int p[MAX], w[MAX], x[MAX];
double maxprofit;
int n, m, i;

void greedyKnapsack(int n, int w[], int p[], int m) {
   double ratio[MAX];
   int j;

   for (i = 0; i < n; i++) {
      ratio[i] = (double)p[i] / w[i];
   }

      for (i = 0; i < n - 1; i++) {
      for (j = i + 1; j < n; j++) {
         if (ratio[i] < ratio[j]) {
            double temp = ratio[i];
            ratio[i] = ratio[j];
            ratio[j] = temp;

            int temp2 = w[i];
            w[i] = w[j];
            w[j] = temp2;

            temp2 = p[i];
            p[i] = p[j];
            p[j] = temp2;
         }
      }
   }

   int currentWeight = 0;
   maxprofit = 0.0;

   for (i = 0; i < n; i++) {
      if (currentWeight + w[i] <= m) {
         x[i] = 1; // Item i is selected
         currentWeight += w[i];
         maxprofit += p[i];
```

```
        } else
        {
            x[i] = (m - currentWeight) / (double)w[i];
            maxprofit += x[i] * p[i];
            break;
        }
    }

    printf("Optimal solution for greedy method: %.1f\n", maxprofit);
    printf("Solution vector for greedy method: ");
    for (i = 0; i < n; i++)
        printf("%d\t", x[i]);
}

void main()
{
    printf("Enter the number of objects: ");
    scanf("%d", &n);

    printf("Enter the objects' weights: ");
    for (i = 0; i < n; i++)
        scanf("%d", &w[i]);

    printf("Enter the objects' profits: ");
    for (i = 0; i < n; i++)
        scanf("%d", &p[i]);

    printf("Enter the maximum capacity: ");
    scanf("%d", &m);

    greedyKnapsack(n, w, p, m);

}
```

OUTPUT:

```
Enter the number of objects: 3
Enter the objects' weights: 10 20 30
Enter the objects' profits: 60 100 120
Enter the maximum capacity: 50
Optimal solution for greedy method: 160.0
Solution vector for greedy method: 1        1        0
```

8. Design and implement C/C++ Program to find a subset of a given set S (sl, s2,…..,sn) of n positive integers whose sum is equal to a given positive integer d

```
#include <stdio.h>

void findSubset(int S[ ], int n, int d, int sum, int index, int subset[ ], int subIndex) {
int I;
   if (sum == d) {
      printf("Subset: ");
      for (i = 0; i < subIndex; i++) {
         printf("%d \t", subset[i]);}
      printf("\n");
      return;
   }

   if (index == n || sum > d)
      return;

   subset[subIndex] = S[index];
   findSubset(S, n, d, sum + S[index], index + 1, subset, subIndex + 1);
   findSubset(S, n, d, sum, index + 1, subset, subIndex);
}

void main()
{
   int n,i, d;
   printf("Enter number of elements (n): ");
   scanf("%d", &n);

   int S[n], subset[n];

   printf("Enter elements S1, S2, ..., Sn: ");
   for ( i = 0; i < n; i++) {
      scanf("%d", &S[i]);}

   printf("Enter target sum (d): ");
   scanf("%d", &d);

   printf("Subsets of S with sum %d:\n", d);
   findSubset(S, n, d, 0, 0, subset, 0);
}
```

OUTPUT:

Enter number of elements (n): 6
Enter elements S1, S2, ..., Sn: 1 2 3 4 5 6
Enter target sum (d): 6
Subsets of S with sum 6:
Subset: 1 2 3
Subset: 1 5
Subset: 2 4
Subset: 6

9. Design and implement C/C++ Program to sort a given set of n integer elements using Selection Sort method and compute its time complexity. Run the program for varied values of n> 5000 and record the time taken to sort. Plot a graph of the time taken versus n. The elements can be read from a file or can be generated using the random number generator.

```
#include <stdio.h>
#include <sys/time.h>

void selsort(int a[], int n) {
    int i, j, small, pos, temp;
    for (j = 0; j < n - 1; j++) {
        small = a[j];
        pos = j;
        for (i = j + 1; i < n; i++) {
            if (a[i] < small) {
                small = a[i];
                pos = i;
            }
        }
        temp = a[j];
        a[j] = a[pos];
        a[pos] = temp;
    }
}

int main() {
    int a[10000], i, n;  // Increased array size for better precision
```

```
struct timeval start, end;  // Structure to store time

printf("\nEnter the n value: ");
scanf("%d", &n);

if (n > 10000) {
    printf("Array size too large, reducing it to 10000.\n");
    n = 10000;  // Limit array size if n is too large
}

printf("\nEnter the array: ");
for (i = 0; i < n; i++) {
    scanf("%d", &a[i]);
}

gettimeofday(&start, NULL);  // Start the timer
selsort(a, n);
gettimeofday(&end, NULL);  // End the timer

// Calculate the time taken in seconds and microseconds
long seconds = end.tv_sec - start.tv_sec;
long microseconds = end.tv_usec - start.tv_usec;
if (microseconds < 0) {
    seconds--;
    microseconds += 1000000;  // Adjust for negative microseconds
}
```

```
    printf("\nTime taken is: %ld seconds and %ld microseconds\n", seconds, microseconds);


    printf("\nSorted array is: ");
    for (i = 0; i < n; i++) {
        printf("%d ", a[i]);
    }
    printf("\n");


    return 0;
}
```

OUTPUT:

Enter the n value: 5

Enter the array: 10 2 3 15 12

Time taken is: 0.000002 seconds

Sorted array is: 2 3 10 12 15

10. **Design and implement C/C++ Program to sort a given set of n integer elements using Quick Sort method and compute its time complexity. Run the program for varied values of n> 5000 and record the time taken to sort. Plot a graph of the time taken versus n. The elements can be read from a file or can be generated using the random number generator**.

```
#include <stdio.h>
#include <stdlib.h>
#include <sys/time.h>

void Exch(int *p, int *q) {
   int temp = *p;
   *p = *q;
   *q = temp;
}

void QuickSort(int a[], int low, int high) {
   int i, j, key;
   if (low >= high)
      return;

   key = low;
   i = low + 1;
   j = high;

   while (i <= j) {
      while (a[i] <= a[key] && i <= high) i++;
      while (a[j] > a[key] && j >= low) j--;
      if (i < j)
         Exch(&a[i], &a[j]);
   }

   Exch(&a[j], &a[key]);
   QuickSort(a, low, j - 1);
   QuickSort(a, j + 1, high);
}

int main() {
   int n, k;
   struct timeval start, end;  // To hold the start and end times
   double ts;

   printf("\nEnter how many numbers: ");
   scanf("%d", &n);
```

```
    // Allocate memory dynamically for the array
    int *a = (int *)malloc(n * sizeof(int));

    // Seed the random number generator
    srand(time(NULL));

    // Generate random numbers
    printf("\nThe Random Numbers are:\n");
    for (k = 0; k < n; k++) {
       a[k] = rand();
       printf("%d ", a[k]);
    }

    // Record the start time using clock_gettime (CLOCK_MONOTONIC is the preferred clock)

    gettimeofday(&start, NULL);  // Start the timer

   QuickSort(a, 0, n - 1);

    gettimeofday(&end, NULL);  // End the timer

long seconds = end.tv_sec - start.tv_sec;

long microseconds = end.tv_usec - start.tv_usec;

if (microseconds < 0) {

   seconds--;

   microseconds += 1000000;  // Adjust for negative microseconds

}


    printf("\nTime taken is: %ld seconds and %ld microseconds\n", seconds, microseconds);



    // Print the sorted numbers
    printf("\nSorted Numbers are: \n");
    for (k = 0; k < n; k++) {
       printf("%d ", a[k]);
    }

    // Print the time taken
    printf("\nThe time taken is %f seconds\n", ts);
```

```
    // Free the dynamically allocated memory
    free(a);

    return 0;
}
```

OUTPUT:

```
Enter How many Numbers: 50

The Random Numbers are:
1804289383 846930886 1681692777 1714636915 1957747793 424238335 719885386 1649760492
596516649 1189641421 1025202362 1350490027 783368690 1102520059 2044897763 1967513926
1365180540 1540383426 304089172 1303455736 35005211 521595368 294702567 1726956429
336465782 861021530 278722862 233665123 2145174067 468703135 1101513929 1801979802
1315634022 635723058 1369133069 1125898167 1059961393 2089018456 628175011 1656478042
1131176229 1653377373 859484421 1914544919 608413784 756898537 1734575198 1973594324
149798315 2038664370
Sorted Numbers are:
35005211 149798315 233665123 278722862 294702567 304089172 336465782 424238335
468703135 521595368 596516649 608413784 628175011 635723058 719885386 756898537
783368690 846930886 859484421 861021530 1025202362 1059961393 1101513929 1102520059
1125898167 1131176229 1189641421 1303455736 1315634022 1350490027 1365180540
1369133069 1540383426 1649760492 1653377373 1656478042 1681692777 1714636915
1726956429 1734575198 1801979802 1804289383 1914544919 1957747793 1967513926
1973594324 2038664370 2044897763 2089018456 2145174067
The time taken is 4.000000e-06
```

11. Design and implement C/C++ Program to sort a given set of n integer elements using Merge Sort method and compute its time complexity. Run the program for varied values of n> 5000, and record the time taken to sort. Plot a graph of the time taken versus n. The elements can be read from a file or can be generated using the random number generator.

```
#include <stdio.h>
#include <stdlib.h>
#include <sys/time.h>

void Merge(int a[], int low, int mid, int high) {
    int i, j, k;
    int b[high - low + 1]; // Dynamic auxiliary array
    i = low;
    j = mid + 1;
    k = 0;

    while (i <= mid && j <= high) {
        if (a[i] <= a[j])
            b[k++] = a[i++];
        else
            b[k++] = a[j++];
    }

    while (i <= mid)
        b[k++] = a[i++];

    while (j <= high)
        b[k++] = a[j++];
```

```
    for (i = low, k = 0; i <= high; i++, k++)
        a[i] = b[k];  // Copy back
}

void MergeSort(int a[], int low, int high) {
    if (low < high) {
        int mid = (low + high) / 2;
        MergeSort(a, low, mid);
        MergeSort(a, mid + 1, high);
        Merge(a, low, mid, high);
    }
}

void  main() {
    int n, a[2000], k;
    struct timeval start, end;
   double ts;

    printf("\nEnter How many Numbers: ");
    scanf("%d", &n);

    printf("\nThe Random Numbers are:\n");
    for (k = 0; k < n; k++) {
        a[k] = rand() % 100; // Random numbers between 0-99
        printf("%d\t", a[k]);
    }
```

```
    gettimeofday(&start, NULL);
    MergeSort(a, 0, n - 1);
    gettimeofday(&end, NULL);

        long seconds = end.tv_sec - start.tv_sec;
    long microseconds = end.tv_usec - start.tv_usec;
    if (microseconds < 0) {
        seconds--;
        microseconds += 1000000;  // Adjust for negative microseconds
    }



    printf("\nSorted Numbers are: \n");
    for (k = 0; k < n; k++)
        printf("%d\t", a[k]);

        printf("\nTime taken is: %ld seconds and %ld microseconds\n", seconds, microseconds);
}
```

OUTPUT:

Enter How many Numbers: 15

The Random Numbers are:

83 86 77 15 93 35 86 92 49 21 62 27 90 59 63

Sorted Numbers are:

15 21 27 35 49 59 62 63 77 83 86 86 90 92 93

The time taken is 3.000000e-06 seconds

12. Design and implement C/C++ Program for N Queen's problem using Backtracking

```
#include <stdio.h>
#include <math.h>
#include <stdlib.h>

int a[30], count = 0;

int place(int pos) {
   int i;
   for (i = 1; i < pos; i++) {
      if ((a[i] == a[pos]) || (abs(a[i] - a[pos]) == abs(i - pos)))
         return 0;
   }
   return 1;
}

void print_sol(int n) {
   int i, j;
   count++;
   printf("\n\nSolution #%d:\n", count);
   for (i = 1; i <= n; i++) {
      for (j = 1; j <= n; j++) {
         if (a[i] == j)
            printf("Q\t");
         else
            printf("*\t");
      }
      printf("\n");
   }
}

void queen(int n) {
   int k = 1;
   a[k] = 0;
   while (k != 0) {
      a[k] = a[k] + 1;
      while ((a[k] <= n) && !place(k))
         a[k]++;
      if (a[k] <= n) {
         if (k == n)
            print_sol(n);
         else {
            k++;
```

```
            a[k] = 0;
         }
      } else {
         k--;
      }
   }
}

void main() {
   int n;
   printf("Enter the number of Queens: ");
   scanf("%d", &n);
   queen(n);
   printf("\nTotal solutions = %d", count);

}
```

OUTPUT:

```
Enter the number of Queens:
4


Solution #1:
*       Q       *       *
*       *       *       Q
Q       *       *       *
*       *       Q       *


Solution #2:
*       *       Q       *
Q       *       *       *
*       *       *       Q
*       Q       *       *

Total solutions = 2
```